

Data Analysis for Experimental Physics with Machine Learning

A.A. 2025/26, Alessandro Foglizzo

Appunti non esaustivi del corso di Data Analysis for Experimental Physics with Machine Learning, tenuto dal prof. Paolo Meridiani presso il dipartimento di Fisica dell'Università di Torino nell'anno 2025/2026.

Concetti fondamentali

- Dataset: insieme di coppie (x_i, y_i) , per $i=1\dots N$ eventi
- Features: insieme di variabili (indice $j=1\dots m$ features) che caratterizzano un evento i : $x_i = (x_{i1}, \dots, x_{ij}, \dots, x_{im})$
- Label: etichetta associata all'evento, è la variabile che lo classifica e viene utilizzata per il training, è il risultato atteso
- Modello: funzione matematica che, prendendo in pasto le features, attraverso i parametri θ ottenuti durante il training, ottiene la predizione su $\hat{y}$
- Funzione oggettiva: $\text{obj}(\theta) = L(\theta) + \Omega(\theta)$, misura il grado di accordo tra il fit e i dati. es MSE
- Loss function $L(\theta)$: misura quanto il modello è predittivo rispetto al training set $L(\theta) = \sum_i (y_i - \hat{y}_i)^2$
- Regularization term $\Omega(\theta)$: controlla la complessità del modello e aiuta ad evitare l'overfitting
- variance-bias tradeoff: equilibrio tra un modello predittivo e semplice
- Training error: $E_{out} = Bias^2 + Var + Noise$
- Varianza: misura quanto cambia la predizione del modello cambiando il dataset di apprendimento, e quindi la sensibilità del modello alle fluttuazioni statistiche del dataset
- Bias: o distorsione, indica la differenza tra le assunzioni sulla semplicità del modello e la complessità dei dati reali, ed è la differenza tra il valore predetto e quello reale
- Overfitting: alta varianza e basso bias, dato da modelli troppo complessi. Errore di training molto basso, errore di test molto alto (non generalizza)
- Underfitting: alto bias e bassa varianza, dato da modelli troppo semplici. Errore di training e di test molto alti.
- Batch: porzione/sottoinsieme del dataset di training
- Iperparametro: configurazione esterna al modello, che non viene appresa ma va fornita prima dell'addestramento
- Epoch: singola iterazione completa su tutto il dataset di addestramento
- Parametri di nuisance: parametri di disturbo, che rappresentano le incertezze sistematiche che possono cambiare la forma o la normalizzazione della pdf. Possono essere di natura teorica o sperimentale

1 Decision Trees

Binary query function $q(x)$ in ogni Branch node, che classifica 0 o 1. Non avendo un numero definito di parametri non si può fare il GD: **Greedy Algorithm**.

Si sceglie lo split massimizzando il delta dell'impurity function $L(R)$:

$$\max_{j,T} \{L(R_p) - (p_P L(R_1) + p_F L(R_2))\}$$

Come impurità si possono usare: Misclassification error, Entropia o Gini index. Per la regressione, si usa il MSE.

Il greedy algorithm non ha problemi di convergenza come il GD. Vediamo le stopping conditions:

- Minimum leaf size
- Effective number of events (per eventi pesati)
- Perfect separation: non si splitta più se in un nodo c'è una sola classe
- Insufficient improvement: stop quando gli split non migliorano la performance (not so good)

Regolarizzazione

Pruning: espandiamo il Tree e a posteriori tagliamo dei sub-tree in base a una loss che penalizza il numero di foglie.

Feature Importance: valutazione del peso delle features nella decisione, per togliere quelle che non contribuiscono e semplificare il modello. $Imp(f) = Q_0 - Q_f$, con Q quality measure calcolata prima con tutte le features e poi rimuovendo una feature f .

Ensembling: combinazione di più modelli per ottenere un modello migliore

Bagging (Bootstrap Aggregating): produce m dataset a partire dal dataset iniziale, creati mescolando sub-dataset con possibile ripetizione. RIDUCE LA VARIANZA.

Random Forest: Bagging + decision trees + more randomness.

Si divide il dataset in tanti subset (bagging) e si costruisce un tree per ogni subset. In ogni nodo del tree vengono usate solo una parte delle features per avere alberi indipendenti tra loro. La decisione finale è il voto pesato di ogni tree.

Boosting: opposto del bagging, riduce il bias di modelli a bassa varianza (weak learners).

Train di modelli in sequenza, sistemando gli errori dei modelli precedenti, poi si prende la media pesata dei modelli come best decision.

Ad es, **Adaboost** assegna i pesi con una loss esponenziale.

Gradient Boosting: simile al Gradient Descent, però nello spazio dei modelli. La loss ha un termine di loss comune e un termine di regolarizzazione.

Segue uno schema additivo, creando un albero a cui fa fare una predizione, e poi crea un altro albero per correggere gli errori del precedente in maniera iterativa. La predizione finale è data dalla somma delle predizioni degli alberi, mentre la funzione oggettiva al passaggio t è data da

$$\text{obj}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) + \text{const}$$

dove l'indice i indica gli eventi.

2 Neural Networks

Estrazione automatica delle features per trovare quelle migliori per un dato task.

Ogni neurone, sommando i pesi associati a ciascuna feature, misura una distanza da un iperpiano che viene non-linearizzata per esaltarne le differenze (4.5).

Diversi tipi di non linearità: Perceptron (θ), sigmoide, tanh, ReLU, Leaky ReLU, ELU (4.6).

L'input dei neuroni del layer denso n ($z = W^T x$) è dato dall'output (attivazioni, $a = \sigma(z)$) dei neuroni del layer $n - 1$, ed ha pesi indipendenti. Tutti i layer, meno l'input e l'output, sono nascosti (hidden): **Feed Forward Fully Connected Neural Network**. L'output dipende dal task:

- Classificazione binaria: output singolo, con attivazione a sigmoide
- Regressione lineare 1D: singolo neurone con attivazione lineare
- Multiclassification: K neuroni con attivazione softmax

Senza le non linearità, i vari strati sarebbero semplificabili a una mera moltiplicazione di matrici, che dà una matrice (lineare).

Universal Approximation Theorem: data una funzione $y = f(x)$ e un $\varepsilon > 0$, esiste un deep network $y = f_w(x)$ di profondità o larghezza arbitraria per cui

$$\sup ||f(x) - f_w(x)|| < \varepsilon$$

Quindi, una NN sufficientemente larga o profonda può approssimare una qualsiasi funzione continua su un dominio compatto. Questo però non garantisce che sia *learnable*.

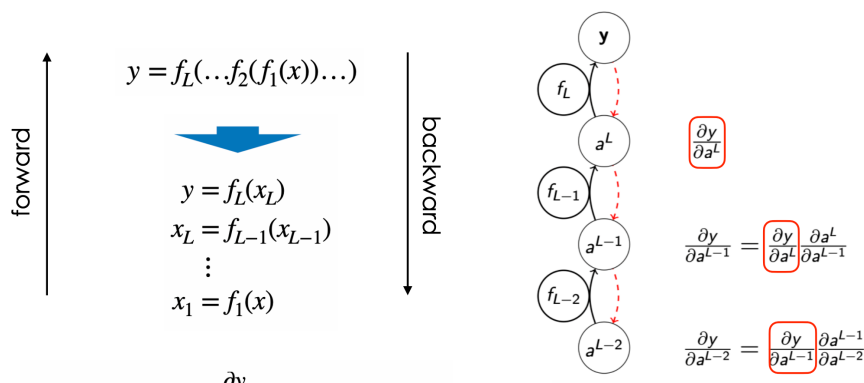
Training NN: a definita la LOSS FUNCTION in base al task e alle assunzioni: **cross-entropy** per classificazione, **MSE** per regressione. Dopodiché, trovare i pesi $\hat{w}$ che massimizzano la massima verosimiglianza $L(w)$. Per il *gradient descent*: $\frac{\partial L(w)}{\partial w}$.

Backward Propagation Algorithm: è l'algoritmo per calcolare i gradienti delle loss function. Sappiamo che che

$$y = f_L(\dots f_2(f_1(x))\dots)$$

con

$$y = f_L(x_L) \text{ e } x_L = f_{L-1}(x_{L-1})$$

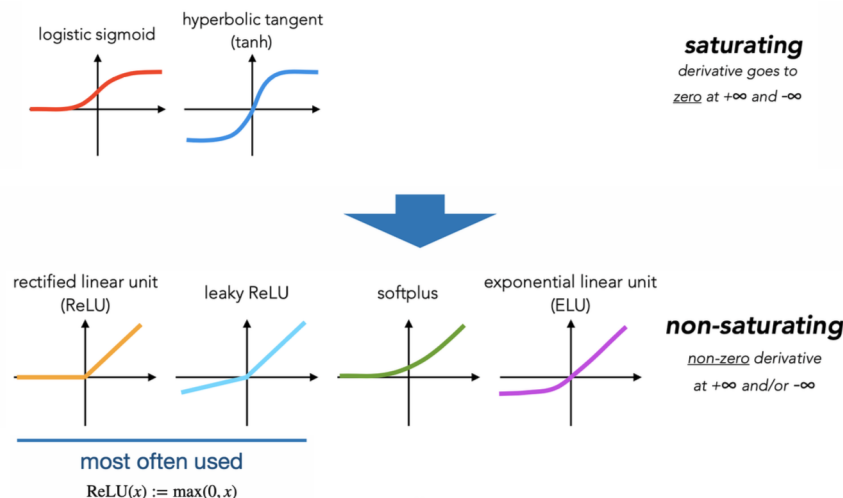


Per calcolare $\frac{\partial y}{\partial x_l}$ con $l \in 1, \dots, L$ dobbiamo applicare la chain rule. Partendo quindi dalla derivata all'output $y = f_L$, calcoliamo a ritroso tutte le $\frac{\partial y}{\partial x_l}$. (4.14)

La backpropagation è fatta in automatico dalla maggior parte dei pacchetti di deep learning. L'unica condizione per poter definire un tipo di layer è saperne definire il gradiente.

Le non linearità *saturating* hanno la derivata molto piccola praticamente ovunque, quindi mandano il gradiente a zero. Inoltre la sigmoide è problematica perché non è centrata in zero e contiene un e^x , lento computazionalmente.

Al contrario, la ReLU è molto usata e fornisce rapida convergenza e computazione; non è però centrata in zero (possibile instabilità, vedere batch normalization).



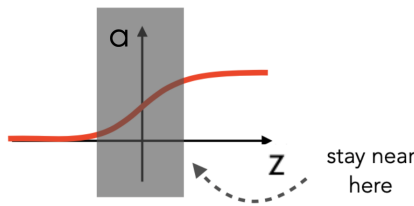
L'obiettivo dell'**inizializzazioni dei pesi** è mantenere attivazioni stabili lungo i vari layer. Vorremmo che $\text{Var}(a) \approx \text{Var}(z)$ per evitare che i segnali esplodano o svaniscano (vanishing gradients) durante la propagazione. Notazione:

- W : matrice dei pesi

- $\mathcal{N}(\mu, \sigma^2)$: distribuzione normale con media μ e varianza σ^2
- $\sim$: estratto dalla distribuzione
- $n_{in/out}$: numero di neuroni in ingresso/uscita del layer (fan in/out)

Esistono diverse inizializzazioni:

- **Zero Initialization:** mette tutti i pesi a zero, da evitare per problemi di simmetria che impediscono l'apprendimento
- **Lecun Initialization:** $W \sim \mathcal{N}(0, \frac{1}{n_{in}})$
- **He (Kaiming) Initialization:** progettata per ReLU e Leaky ReLU, $W \sim \mathcal{N}(0, \frac{2}{n_{in}})$ (il fattore 2 compensa l'azzeramento, in media, della metà delle attivazioni)
- **Xavier (Glorot) Initialization:** progettata per sigmoide e tanh, $W \sim \mathcal{N}(0, \frac{1}{n_{in}+n_{out}})$ (cerca un bilanciamento tra il passo avanti e quello indietro)



Batch Normalization: trasformazione che riporta gli input delle attivazioni ad avere media 0 e varianza 1 e quindi rimanere all'interno del range dinamico della funzione non lineare. Si calcola la media μ_B e la varianza σ_B^2 delle attivazioni all'interno di ogni batch, dopodiché si introducono due parametri apprendibili: una scala γ e uno spostamento β .

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

La Batch Normalization sembra funzionare perché rende la loss function più smooth nei parametri.

NB: utilizzando la ReLU, è comune applicare la BN **dopo** l'attivazione, perché la ReLU rimuoverebbe i valori negativi.

Connettività Residuale: rispetto a una rete con connettività sequenziale, in cui l'informazione deve passare attraverso tutti i layer per arrivare all'output e ogni layer trasforma completamente l'input ricevuto, questa permette di trovare dei percorsi alternativi più brevi. Invece di imparare una funzione complessa $H(x)$, si parte da un input x e si impara solo la funzione differenza tra $H(x)$ e x . Matematicamente, $H(x) = F(x) + x$. Se x è già una buona approssimazione di $H(x)$, la rete dovrà solo imparare una piccola correzione $F(x)$. In questo modo, il termine $+x$ evita che i gradienti vadano a zero e permette una backpropagation più fluida; inoltre, l'informazione è preservata anche quando $F(x)$ è molto piccola.

Learning Rate α : si dovrebbe iniziare l'apprendimento con un learning rate alto, in modo da convergere più rapidamente ed evitare minimi molto pronunciati. Dopodiché, per convergere a un risultato va diminuito il learning rate, ma non troppo presto e in più fasi in modo da evitare di finire in un minimo locale. Si possono adottare diverse strategie:

- Exponential Decay: decadimento graduale nel tempo, $\alpha_t = \alpha_0 \exp^{-\lambda t}$
- Cosine Annealing: riduzione graduale con una funzione coseno, $\alpha_t = \frac{1}{2} \alpha_0 (1 + \cos(\frac{t}{T} \pi))$
- 1Cycle Policy: aumenta il learning rate all'inizio, poi decade per il fine tuning

Riduzione dell'Overfitting (Regolarizzazione): tecniche di regolarizzazione per ridurre l'errore di generalizzazione limitando la complessità del modello e restringendo lo spazio dei parametri.

- Early Stopping: interrompe l'addestramento se la rete non è migliorata per n epochs. Iperparametro: *patience*.
- Dropout: impostazione di alcuni neuroni a zero, scelti randomicamente con probabilità p (iperparametro). I pesi per la predizione vanno riscalati di un fattore p , ma è fatto in automatico dalle librerie di ML: $w_{test}(p) = p w_{train}(p)$

Consiglio: monitorare sempre la training history, controllando le figure di merito (loss, accuracy, MSE) per i dataset training e validazione. Inoltre, evitare numeri molto piccoli e molto grandi e controllare se ci sono NaN quando il modello non migliora.

3 Calibrazione della risposta di un rivelatore

Con **risposta di un rivelatore** ci si riferisce alla misura ottenuta da un rivelatore che ha ricevuto uno specifico input. I rivelatori non producono misure esatte e per uno stesso input fisico y l'output x potrebbe variare per effetti stocastici (noise e risoluzione) o sistematici (soglie di misurazione,...). Quindi, la risposta x del rivelatore è governata da una legge di *probabilità condizionale* $p(x|y)$.

La calibrazione del detector si può formalizzare come il problema inverso, $p(y|x)$.

Features in input: osservabili x del rivelatore (energia ricostruita, posizione, forma degli sciami,...)

Target: quantità y (energia true o θ_{true} , normalmente E_{true}/E_{reco} o $\theta_{true} - \theta_{reco}$).

Questa ricostruzione subisce tutti gli effetti del detector: noise, risoluzione, bias, non linearità. Parametrizzare $p(y|x)$ è spesso complesso, il goal è apprendere questa funzione per caratterizzare il comportamento del detector e correggere i bias.

Una grossa sfida è data dalle distribuzioni della risposta: non gaussiane, asimmetriche e con lunghe code. Quindi la media potrebbe non corrispondere al valore più probabile.

b-jet: gli adroni con b decadono quasi sempre in adroni c , in parte con canali leptonici, e quindi aggiungono particelle leggere (neutrini, muoni, u/d-jets) che finiscono "fuori dal cono". Per questo, è necessaria una calibrazione specifica. per i b-jet

4 Classificatori per analisi di eventi HEP

Bosone di Higgs: $m_H = 125$ GeV, $\tau_H \sim 10^{-25}$ s, scalare.

Produzione: accoppiamento con fermioni (in particolare t che ha la massa, e quindi l'accoppiamento con l'Higgs, maggiore) ggH (gluon fusion con loop t 90%) e ttH ($2g \rightarrow t\bar{t}t\bar{t}$); accoppiamento con bosoni vettori VBF (vector boson fusion, $2q \rightarrow qqWW/qqZZ$), VH ($q\bar{q} \rightarrow W/Z \rightarrow W/Z+H$).

Decadimento nelle **Higgs Big Five**:

Channel	$\sigma(m_H)/m_H$ [%]	m_H range [GeV]	S/B (typ.)
$H \rightarrow ZZ \rightarrow 4\ell$	1–2	110–1000	High (~ 1)
$H \rightarrow \gamma\gamma$	1	110–150	Moderate ($\sim 0.05\text{--}0.2$)
$H \rightarrow WW \rightarrow 2\ell 2\nu$	20	110–660	Low (~ 0.01)
$H \rightarrow \tau\tau$	15	110–145	Low (~ 0.02)
$H \rightarrow b\bar{b}$	10	110–135	Very low ($\sim 0.001\text{--}0.01$)

I canali con migliore risoluzione di massa e più grande S/B sono i più adatti per scoprire e misurare il bosone di Higgs.

Flow di un'analisi in HEP

1. Definire che cosa si vuole misurare
2. Scegliere uno stato finale (o canale)
3. Identificare i processi che compongono il fondo
4. Definire come selezionare gli eventi
5. Definire un'osservabile (numero di eventi, massa invariante, asimmetrie,...) con istogrammi
6. Ottenere le misure e le loro incertezze

Analisi $H \rightarrow \gamma\gamma$

Ricerca di un picco piccolo e stretto su un grande fondo che decade esponenzialmente. È necessaria una buona risoluzione energetica e spaziale del calorimetro EM per determinare l'angolo tra i due fotoni.

Bump-Hunt Analysis: il fondo che decade permette una stima robusta del fondo direttamente nella zona del segnale. In questo caso, su uno spettro di massa invariante $m_{\gamma\gamma}$.

$$m_{\gamma\gamma} = \sqrt{2E_1 E_2 (1 - \cos \theta_{12})}$$

Simple-cut analysis: selezione degli eventi che contengono 2 fotoni isolati con $p_T > 30$ GeV e $|\eta| < 2.5$: definisce una regione di segnale usando tagi rettangolari sulle features osservabili (p_T, η , isolation).
 Detta σ_m la risoluzione del picco della massa, il rapporto S/B è proporzionale a $1/\sigma_m$: migliorare la risoluzione della massa porta direttamente a una migliore separazione del segnale dal rumore. Questo si può fare con una regressione energetica (DT o NN) per correggere la risposta dell'ECAL, oppure con una stima di σ_m evento per evento, assegnando poi i pesi maggiori agli eventi con miglior risoluzione.
 Un'altra via per massimizzare S/B è migliorare la selezione degli eventi, cioè ridurre il False Positive Rate (FPR) mantenendo lo stesso True Positive Rate (TPR).

- identificazione di fotoni isolati, per ridurre i fake-photons
- cinematica del sistema di-photon: usare la correlazione tra i due fotoni per separare $H \rightarrow \gamma\gamma$ dagli eventi di QCD
- evitare che l'output dei classificatori sia correlato alla massa invariante dei due fotoni, per mantenere il "bump-hunt"

Di-photon classifier: Boosted Decision Tree (BDT) addestrato su campioni simulati di segnali di Higgs (attraverso tutti i canali di produzione) e di fondo (mix di di-photons di QCD, γ +jet e di-jet fake).
 Per evitare che il classificatore impari ad usare la massa per discriminare i segnali, dobbiamo dare in input features che siano indipendenti da $m_{\gamma\gamma}$.

1	$p_T^1/m_{\gamma\gamma}$	Mass independent kinematics
2	$p_T^2/m_{\gamma\gamma}$	
3	η_1	
4	η_2	
5	$\cos \Delta\phi_{\gamma\gamma}$	
6	$\sigma_m/m_{\gamma\gamma}$ (Right Vtx Hypothesis)	Mass resolution
7	$\sigma_m/m_{\gamma\gamma}$ (Wrong Vtx Hypothesis)	
8	p_{vtx}	
9	IDMVA ₁	Photon Identification score
10	IDMVA ₂	

Categorizzare gli eventi in base alla purezza del segnale migliora la sensibilità dell'intera analisi.

Forza del segnale: (6.16 per la funzione di likelihood con i parametri) Nel fit della funzione prevista dal Modello Standard si aggiunge un parametro di **forza del segnale** μ su cui si fitta: se $\mu = 1$ dal fit, i dati sono in perfetto accordo, se $\mu = 0$ non c'è segnale. Viene aggiunto nella forma $f \propto \mu s(x; \theta) + (x; \theta)$, dove s e b indicano signal e background.

Per tenere conto dei possibili errori sistematici (teorici e sperimentali, che possono cambiare la forma della distribuzione: fondo, luminosità, calibrazione) si aggiunge un insieme di parametri di *nuisance* θ .

Data la funzione di massima verosimiglianza globale $\mathcal{L}(\mu, \theta)$, $\hat{\mu}$ e $\hat{\theta}$ sono i parametri che massimizzano $\mathcal{L}$: sono Maximum Likelihood Estimator (MLE).

Con i parametri di nuisance si può inoltre fare la **profilazione** (profiling): il set di parametri $\hat{\theta}(\mu)$, detti MLE condizionali, sono i valori che massimizzano $\mathcal{L}$ per un certo valore fissato di μ .

Si definisce il **Profile Likelihood Ratio**

$$\lambda(\mu) = \frac{\mathcal{L}(\mu, \hat{\theta})}{\mathcal{L}(\hat{\mu}, \hat{\theta})}$$

sempre compreso tra 0 e 1, in quanto il denominatore è un massimo assoluto e il numeratore un massimo vincolato. Se il valore di μ scelto nella profilazione è in forte accordo con i dati ottenuti (e quindi con $\hat{\mu}$), $\lambda \rightarrow 1$, se invece è in disaccordo $\lambda \rightarrow 0$.

Statistica di test: per effettuare i test statistici che servono a rigettare o accettare un'ipotesi, l'intero dataset viene ridotto alla variabile di test

$$q(\mu) = -2 \ln(\lambda(\mu))$$

che trasforma dei piccoli valori di probabilità tra 0 e 1 in valori positivi più facili da gestire: grandi valori di q indicano forte disaccordo tra ipotesi e dati.

Per misure di forza del segnale, la massa m_H viene trattata come gli altri parametri di nuisance e profilata; quando si fanno misure di massa, è la forza del segnale ad essere profilata.

Robust Classifier: si può sviluppare un modello di ML perché sia indipendente (insensibile) rispetto a una specifica variabile. Questa può essere una variabile fisica (come massa invariante o momento trasverso) o un parametro di nuisance.

Nel caso di un parametro fisico come la massa invariante, è importante evitare che la selezione di eventi scolpisca artificialmente la forma del fondo, e mantenere l'indipendenza permette di stimare il fondo a partire dai dati reali.

Nei modelli di simulazione MC alcuni parametri (come pileup, material budget, calibrazione) potrebbero essere fissati: se nei dati reali alcuni di questi valori fossero diversi, il modello ML perderebbe potere discriminativo. Questa situazione, in cui simulazione e test hanno PDF diverse, è detta *Domain Adaptation* e ci sono tre tecniche per approcciarla:

- Calibrazione dell'output: si addestra il classificatore sul dominio di training, e poi si seleziona un campione del dataset di test per confrontare le prestazioni e ricalibrare.
- Approccio parametrizzato: si parametrizza completamente l'andamento rispetto a una variabile di nuisance α , che poi viene usata come feature di input del modello. Bisogna conoscere a priori una PDF per applicarci la nuisance.
- Approccio Robust Classifier: rendere il classificatore indipendente dalla nuisance; può essere meno performativo dell'approccio parametrizzato ma è più utile in caso non fosse pratico modellizzare le pdf.

Tre possibili approcci per ridurre la dipendenza dalla nuisance: planing, decorrelating loss penalties, adversarial training.

Planing: preprocessamento dei dati per ridurre o rimuovere la dipendenza dello score del classificatore da α . Si implementa evitando che il classificatore consideri la feature o il parametro di nuisance come oggetto discriminante tra diverse ipotesi di classe. Nel *planing*, se vogliamo indipendentizzare dalla massa invariante M , si pesa ogni evento di training con $w(M)$ in modo che in funzione di M le PDF del segnale e del fondo siano uguali. Date p_S e p_B le pdf per segnale e fondo, avremo i pesi:

$$\text{segnale: } w(M) = \frac{1}{p_S(M)}, \quad \text{fondo: } w(M) = \frac{1}{p_B(M)}$$

I pesi entrano nel calcolo del loss nel training ma non nella validation o nel testing.

Limitazione: se ci sono altre features che potrebbero correlare il classificatore, lo informeranno sul valore della variabile "planata".

Decorrelating Loss Penalties: si aggiunge un termine di penalizzazione ($d\text{Corr}^2$) alla funzione di perdita, che misura la dipendenza non lineare tra l'output della rete e la variabile che si vuole decorrelare, penalizzando il modello se l'indipendenza non è rispettata. È pesata da un iperparametro λ . La Distance Correlation (DisCo, $d\text{Corr}^2$) si definisce a partire dalla *covarianza di distanza* ($d\text{Cov}^2$), che misura qualsiasi correlazione anche non lineare.

$$d\text{Cov}^2(X, Y) = \langle |X - X'| | Y - Y' \rangle + \langle |X - X'| | Y - Y'' \rangle - 2 \langle |X - X'| | Y - Y' \rangle$$

$$d\text{Corr}^2(X, Y) = \frac{d\text{Cov}^2(X, Y)}{\sqrt{d\text{Cov}^2(X, X) \cdot d\text{Cov}^2(Y, Y)}}$$

dove gli apici indicano copie i.i.d. (ovvero coppie diverse di punti estratte dallo stesso dataset).

La loss function si può quindi scrivere come

$$\mathcal{L} = \mathcal{L}_{\text{classifier}}(\vec{y}, y_{\text{true}}) + \lambda \, d\text{Corr}_{y_{\text{true}}=0}^2(\vec{m}, \vec{y})$$

dove $\vec{y}$ è il vettore degli output (predizioni) della rete, e $\vec{m}$ il vettore della variabile da cui ci si vuole decorrelare (in questo caso la massa). Il calcolo di $d\text{Corr}^2$ per $y_{\text{true}} = 0$ indica che va calcolata solamente per gli eventi di fondo.

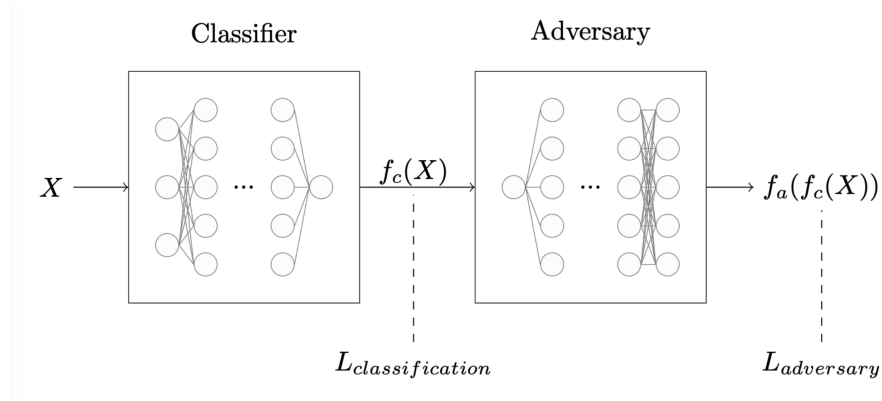
Adversarial Training: è un'estensione della penalized loss, in quanto la loss function è la combinazione di due parti. Alla loss standard, si aggiunge una loss avversaria (modulata da un iperparametro) che funziona da penalty.

La loss è formata da due NN che competono tra loro:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{cls}}(\theta) - \lambda \cdot \mathcal{L}_{\text{adv}}(\phi; \theta)$$

in cui si minimizza $\mathcal{L}_{\text{total}}$ rispetto a θ e $\mathcal{L}_{\text{adv}}$ rispetto a ϕ . In altre parole,

$$\min_{\theta} \max_{\phi} [\mathcal{L}_{\text{cls}}(\theta) - \lambda \cdot \mathcal{L}_{\text{adv}}(\phi; \theta)]$$



Il compito della rete avversaria è di imparare $p(z|F_c(X))$, quindi indovinare il valore della variabile di disturbo z a partire dall'output $F_c(X)$ del classificatore. Ma il classificatore verrebbe penalizzato se il suo output permettesse all'avversaria di ottenere questo risultato. È un training delicato perché la loss totale non è più limitata inferiormente.

NN Parametrica: si usa un parametro come feature di input e si addestra usando dataset generati con specifici valori del parametro. Se questo non è definito per una classe (eg la massa nel sample del fondo) si usa un valore casuale.

Vantaggi: permette di interpolare tra i valori dei parametri e ha una performance migliore rispetto a una sola rete addestrata su un sample con parametri mischiati.